Performance Engineering

Analysis and Optimisation of the CloverLeaf Mini-App on the HAICGU Kunpeng 920 Cluster

Jort Oosterveld S5545781 & Sergiu-Adrian Preda S6778542
Group 11

Draft, October 19, 2026

1 Introduction

CloverLeaf [2] solves the compressible Euler equations on a 2D structured grid with an explicit Lagrangian–Eulerian scheme; it is a stencil code and a standard case for memory-bound performance engineering [1]. We work on the reference implementation (`CloverLeaf_ref`, commit `b5e598d`, MPI+OpenMP) and optimise its *C kernels* (`use_c_kernels`) on one node of the HAICGU cluster. Section 2 establishes where the baseline stands against the hardware, Section 3 derives the kernel changes that can still pay off and shows them in code, and Section 4 compares the optimised kernels with the baseline at every layout.

2 Baseline Analysis

2.1 Application and system

One time step runs, in order, the equation of state (`ideal_gas`), artificial viscosity, the stable time step (`calc_dt`), a predictor and a corrector PdV step, the node acceleration, the volume fluxes, the advection of cell quantities (`advec_cell`) and of momentum (`advec_mom`) in two directional sweeps, and the reset of the time levels. Every kernel is a loop nest over the grid with a ± 1 stencil; the two advection kernels contain an upwind/limiter loop with data-dependent indices. All fields are double-precision arrays of $(n_x + 5) \times (n_y + 5)$ elements. The benchmark problem is **bm3840**: 3840^2 cells, 87 time steps, about 25 arrays of 118 MB ($\approx$3 GB working set, far beyond the 2×64 MB L3), with CloverLeaf's built-in kinetic-energy check (`PASSED` in every run reported here).

The node has 2× Kunpeng 920 (ARMv8, 2.6 GHz, 64 cores each, 4 NUMA domains of 32 cores) and 16 channels of DDR4-2666 (128 GB, 341 GB/s theoretical), under a RHEL-8-compatible Linux with 64 KB pages. Software: GCC/GFortran 14.1.0, OpenMPI 5.0.3, PAPI 7.1.0 and Linux `perf`; the kernels are compiled with the project's `GNU` settings, `-O3 -march=native -funroll-loops -fopenmp`. Nodes are used exclusively. A configuration "$r \times t$" is r MPI ranks with t OpenMP threads each. Every rank is bound to t consecutive cores (`-map-by slot:PE=`t) and OpenMP pins one thread per core (`OMP_PLACES=cores`, `OMP_PROC_BIND=close`). The Fortran kernels are run at the same configurations for reference only.

kernels	build	runs	deviation (%)	layouts (all identical)	gate
c	base	22	0.5726209E-02	128x1, 4x32, 64x2, 1x128, 32x4	ok
fortran	base	12	0.1787088E-01	128x1, 4x32, 64x2	ok
c	opt	6	0.3835202E-01	64x2, 1x128, 4x32, 32x4, 128x1	MISMATCH
fortran	opt	1	0.1787088E-01	64x2	ok
c	flags	1	0.3835202E-01	64x2	MISMATCH
c	cell	1	0.3835202E-01	64x2	MISMATCH
c	mom	1	0.3835202E-01	64x2	MISMATCH
c	optnf	6	0.5726209E-02	64x2, 1x128, 4x32, 32x4, 128x1	ok
fortran	optnf	1	0.1787088E-01	64x2	ok

Table 1: Kinetic-energy deviation of `clover_bm.in`: the baseline value an optimised build has to reproduce (jobs 34729 and 34796, re-confirmed by the rebuilt baseline binary of the ladder in job 34999) and the value every build with the compiler flags produced (job 34937, one run per layout). The gate column is the verdict of `qa_check.py` against the baseline value; see Section 4.

2.2 Methodology

Tools. Three independent sources: (1) CloverLeaf's internal profiler (wall clock per algorithmic phase); (2) PAPI per kernel, through a small instrumentation layer (`papi_prof`) that wraps the `omp parallel` region of each C kernel, keeps one event set per thread and writes the counts per thread at exit (compiles to nothing without `-DUSE_PAPI`); cycles/instructions, instruction mix and stall sets are measured in separate runs; (3) Linux `perf`: sampling profiles, whole-run counters, and the HiSilicon DDR-controller PMU (`hisi_sccl*_ddrc*/flux_rd,wr`), read system-wide around every run, which counts the bytes that cross the memory interface per controller.

Protocol. Five repetitions per configuration, median and min–max reported; the timed region is the complete run of 87 steps including start-up; every repetition runs under the DDR-controller counters, so wall clock and hardware bytes belong to the same run. All bandwidths are *hardware bytes* (DDR-controller counts × 32 B) over wall clock, for STREAM and for the application alike. The harness (`perf/baseline`) is driven by matrix files and environment variables and runs through `mpirun` only.

Checks of the tools. The PAPI build costs nothing measurable (11.30 s against 11.34 s at 64×2; 521–523 s against 528 s on one core) and its per-kernel times add up to the wall clock within 1%; `MEM_ACCESS` equals `PAPI_L1_DCA` in every kernel; the cycle counter gives 2.59 GHz; the DDR counters were calibrated on STREAM (Section 2.4). The placement of every multi-rank run is checked by the counters: the busiest controller group carries 1.00× the mean.

Correctness gate. CloverLeaf's own check on the long deck `clover_bm.in` (960^2 cells, 2955 steps, test problem 3) reports a deviation of 0.0057% (C) and 0.018% (Fortran) from the reference compiled into the code even for the *unmodified* kernels, identical to all digits at every layout and repetition (Table 1): the reference was produced on another machine. Every modified kernel therefore has to reproduce this baseline deviation to 10^{-6}% (`perf/baseline/qa_check.py`) and print `PASSED` on bm3840.

2.3 Where the time goes

Table 2 gives the time share and the counters of every kernel. The two advection kernels take 48% of the time on one core and 63% on 128; `advec_mom` scales worst of the large kernels (14× against 22–59×). On one core the IPC is 0.35–1.07 and 60–87% of the cycles are back-end stalls; at 64×2 the IPC is 0.10–0.50 per kernel and the stall share 81–96%: the cores wait for

#	kernel	time share (%)		IPC	vector	FP scalar	back-end
		1 core	128 cores	1 core	ins. (%)	ins. (%)	stall (%)
1	`advec_mom`	28.6	44.0	0.85	16	17	71 / 94
2	`advec_cell`	19.4	19.4	0.75	12	25	73 / 93
3	`calc_dt`	11.8	4.8	0.65	0	64	75 / 84
4	`pdv_correct`	9.0	5.0	0.35	62	0	83 / 93
5	`pdv_predict`	8.6	4.6	0.35	60	0	87 / 95
6	`ideal_gas`	7.4	4.7	0.52	0	53	79 / 92
7	`viscosity`	5.7	2.2	1.06	0	64	60 / 79
8	`accelerate`	5.2	4.1	0.63	41	0	77 / 93
9	`flux_calc`	2.2	3.0	0.53	45	0	80 / 96
10	`reset_field`	1.1	3.2	0.56	0	0	80 / 97
11	`revert`	0.5	1.3	0.64	0	0	78 / 96
12	`field_summary`	0.4	0.2	0.64	56	24	72 / 90
13	`update_halo`	0.1	3.5	0.67	3	0	49 / 95

Table 2: Per-kernel time share (bm3840; "128 cores" = 1×128), IPC, share of SIMD and scalar floating-point instructions, and back-end stall cycles on 1 / 128 cores. Numbers refer to Figure 1. `reset_field`, `revert` and `update_halo` do no floating-point work.

memory. Three kernel groups appear in the instruction mix: `pdv`, `accelerate` and `flux_calc` are vectorised (41–62% NEON); `calc_dt`, `ideal_gas` and `viscosity` run entirely scalar (they contain `sqrt`, which GCC keeps scalar without `-fno-math-errno`) but are only 12% of the node-level run; `advec_mom` and `advec_cell` are 12–16% vector instructions because their upwind loops have data-dependent indices. The `perf` profile of a 128-thread run adds that 12.5% of all cycles are spent in the OpenMP barrier (`gomp_team_barrier_wait_end`): every kernel is a sequence of worksharing loops with an implicit barrier each.

2.4 Ceilings and byte model

Memory. STREAM triad (10^8 doubles, 20 iterations, 10 runs per configuration) sustains 15 GB/s on one core, 67 GB/s per NUMA domain (32 threads) and **266.5 GB/s** for the node, in every one of 10 pinned repetitions. Under the DDR-controller counters the controllers move 1.04× the bytes STREAM counts itself (24 B per triad element) and 0.74× the write-allocate model, in all 20 STREAM runs and in our own triad kernel: a streaming store does *not* read the line first on this CPU, one count is 32 B, and the hardware ceiling is the counted triad, 78% of the bus. (The DIMM clock is not readable without root; 266.5 GB/s over 16 channels needs at least 2082 MT/s, so DDR4-2666 is consistent.)

Compute. Independent FMA chains on 2-lane NEON registers reach 4.0 flop/cycle per core and **1330 GFLOP/s** per node; scalar FMA reaches the same, so vectorisation saves instructions, not flops. Vector division runs at 0.19 GFLOP/s per core. The ridge point of the node is 1330/266.5 = 5 flop/B (0.7 flop/B for one core).

Byte model. For every loop nest of the C kernels we counted the arrays read and written and the flops per cell; each array is streamed from DRAM once per loop nest, and a store is charged a read-for-ownership (write-allocate) in addition to the write. Table 3 gives 451 flop and 2186 B per cell and step (1688 B without the read-for-ownership), an intensity of 0.21 flop/B.

Validation. The model predicts 2.80 TB for bm3840; the counters measure 2.83 TB at 64×2 in all five repetitions (1% above), 2.79–2.99 TB at the other layouts. The read/write split decides which version of the model applies: the counters see 2.05 TB of reads and 0.77 TB of writes against 1.53 TB of demand reads and 0.64 TB of writes in the model, i.e. the reads are 1.34× the

kernel	calls/step	flop/cell	B/cell (no WA)	B/cell (with WA)	AI (flop/B, WA)
`ideal_gas`	2.1	9	32	48	0.19
`viscosity`	1	40	40	48	0.83
`calc_dt`	1	31	80	88	0.35
`pdv_predict`	1	33	96	120	0.28
`pdv_correct`	1	33	112	136	0.24
`revert`	1	0	32	48	0.00
`accelerate`	1	37	80	96	0.39
`flux_calc`	1	10	64	80	0.12
`advec_cell`	2	45	180	244	0.18
`advec_mom`	4	39	172	220	0.18
`reset_field`	1	0	64	96	0.00
`field_summary`	0.1	22	48	48	0.46
per cell and step		451	1688	2186	0.21

Table 3: Analytic flop and byte counts per cell and call (WA = write-allocate). `ideal_gas` runs 2.1 times per step.

demand reads. Unlike STREAM, the C kernels' stores *do* cause a read of the line first, for about 70% of the written volume; the write-allocate model is the right one for the code as it stands. The Fortran build writes the same amount (0.60 TB) but reads only 1.15× the demand reads (1.76 TB, 2.36 TB in total): its loops avoid most of that traffic, and this 0.47 TB is the entire C/Fortran gap (Section 2.5).

2.5 Baseline performance

Table 4 gives the layouts of the C kernels with the hardware bandwidth they sustain over the whole run, start-up included. **64×2 is the baseline**: 11.3 s (spread 0.8%), 2.83 TB, 250 GB/s = **94% of the hardware ceiling**; 32×4 and 128×1 reach 91–95%, 4×32 83%. Transparent huge pages (512 MB on this system) matter only for the single-rank 1×128 layout, where first touch places whole arrays on one NUMA domain and the busiest controller group carries 1.66× the mean (26.9 s, against 13.3 s with THP disabled for the process); with four or more ranks every rank touches its own data and THP has no effect, so the baseline needs no change to the system.

Figure 1 places every kernel on the rooflines of one core and of the node. Every kernel lies at 0.13–0.83 flop/B, far left of the 5 flop/B ridge; even `viscosity` could reach only 17% of the FMA peak with perfect bandwidth. In hardware-byte terms (per-section roofline of the 64×2 run, `perf/roofline/roofline.py -hw`) both advection sections, PdV, `ideal_gas` and the fluxes run at 94–102% of the memory roof, acceleration at 83%, and the scalar `sqrt` kernels at 63–66%. The floor at the measured traffic is 2.83 TB / 266.5 GB/s = **10.6 s**; the baseline is within 7% of it. There is no headroom at the current byte count: *every further gain has to come from moving fewer bytes*, and the Fortran build shows that this is possible (9.33 s, 2.36 TB, at the same 252 GB/s).

kernels	layout	wall (s) median (min–max)	DRAM (TB)	bandwidth (GB/s)	busiest/ mean	vs. hw roof (%)
c	64x2	11.32 (11.24–11.33)	2.83	250	1.00	94
c	1x128	26.91 (17.44–29.20)	2.99	111	1.66	42
c	4x32	13.00 (12.76–13.21)	2.87	221	1.01	83
c	32x4	11.46 (11.41–11.47)	2.79	244	1.00	91
c	128x1	11.66 (11.63–11.77)	2.95	253	1.00	95
c	1x128, THP off	13.31 (13.28–13.92)	2.83	212	1.00	80
fortran	64x2	9.33 (9.33–9.42)	2.35	252	1.00	95
fortran	1x128	20.63 (18.40–25.92)	2.44	118	1.45	44
fortran	4x32	10.88 (10.78–11.22)	2.38	219	1.00	82
fortran	32x4	9.46 (9.44–9.52)	2.30	243	1.00	91
fortran	128x1	9.76 (9.66–9.77)	2.46	252	1.00	94

Table 4: Baseline layouts, bm3840, job 34797: median of 5 (min–max), hardware bytes of the DDR-controller counters, the resulting bandwidth and its share of the 266.5 GB/s ceiling, and the load of the busiest controller group relative to the mean. THP as installed except for the one marked row.

3 Kernel Improvements

3.1 What the analysis allows

The baseline sustains 94% of the memory ceiling, so instruction-side work (vectorising the upwind loops, `-fno-math-errno` for the `sqrt` kernels, fewer barriers) cannot pay off until the bytes shrink; those items are kept as time-only changes (Section 3.4). The byte count has two components we can change: the bytes the algorithm asks for (temporaries, dead stores, copies, repeated helper loops) and the read-for-ownership that the C stores pay and a streaming store would not. Every byte change below is expressed as an edit of the loop table of the byte model (`perf/analysis/plan_bytes.py`), so the prediction and the later measurement use the same convention, and every change has to pass the correctness gate of Section 2.2.

3.1.1 Selection

The original plan was to take the three kernels with the largest theoretical gain in the byte model and leave the rest. These are: `advec_mom`, whose helper loops are recomputed for every velocity component. `reset_field` and `revert`, which solely copy data, and `advec_cell`, which round-trips four temporaries through memory. While reading these kernels, further opportunities presented themselves and were implemented as well: two dead stores (`post_vol`, `pre_vol`), the fused reduction of `calc_dt`, two compiler flags that unblock vectorisation, `nowait` between loops that are provably independent, and a branch-free formulation of the `advec_mom` upwind loop. Changes (1)–(6) below reduce bytes, (T1)–(T4) reduce time only; Section 3.5 gives the state of each.

3.2 Flop and byte counts per cell

The counts of Table 3 were made by reading every loop nest of `kernels/*_kernel_c.c` and listing: the 2D arrays it reads, the 2D arrays it writes and the floating-point operations per cell. The bandwidth of bytes per cell of a loop nest is

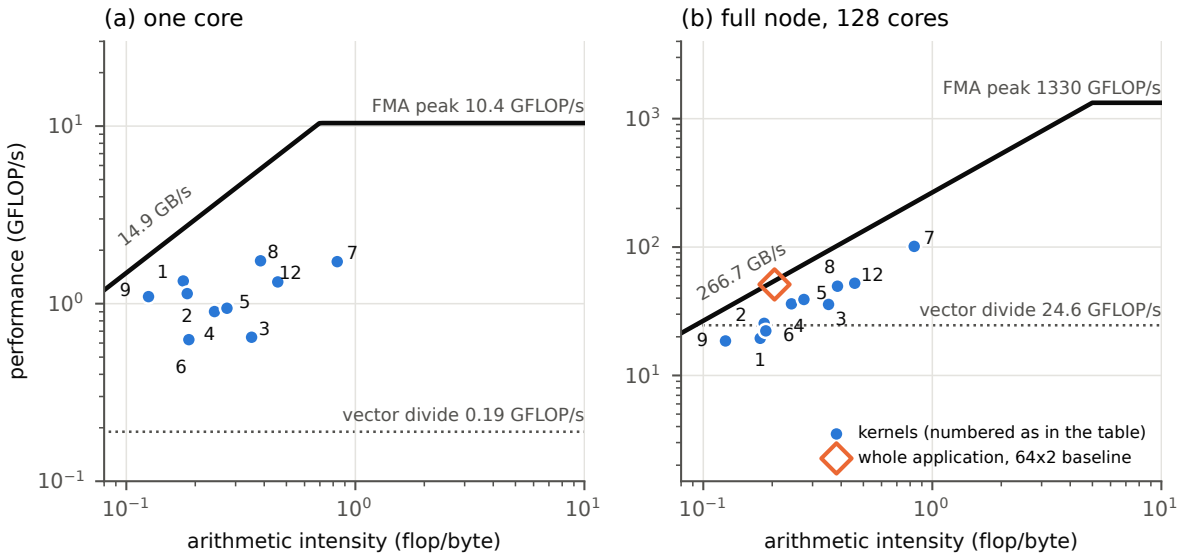


Figure 1: Roofline of the CloverLeaf C kernels (bm3840), one core and full node. Kernel numbers refer to Table 2; kernel points use model bytes and PAPI times (1×128), the application point the measured DRAM bytes and wall clock of the 64×2 baseline.

$$8\,\mathrm{B} \times (\text{arrays read} + \text{arrays written} + \mathrm{WA} \times \text{arrays written but not read})$$

Every array is streamed from DRAM once per loop nest, since it easily large enough not to fit in cache. Stencil neighbours $j \pm 1$, $k \pm 1$ hit in cache (a few rows of 3840 doubles fit in the 512 KB L2), and the read-for-ownership (WA) is charged once for every array a loop stores without loading. 1D arrays (`celldx`, `vertexdx`, ...) and halo cells are ignored. Flops: $+$, $-$, $\times$, $/$ and `sqrt` count one; comparisons, `fabs`, `MIN`, `MAX` and `SIGN` count zero; loop-invariant constants are hoisted. Kernels whose loops differ between the two sweeps are averaged over the sweeps. Per kernel and call:

- `ideal_gas`: reads density and energy, writes pressure and soundspeed: four streams, two of them write-allocated. Nine flops: $1/\rho$, 0.4ρ, the product with e, ρp, the sound-speed expression $v^2(p\,p_e - p_v)$ and one `sqrt`.
- `viscosity`: four reads (both velocities, pressure, density), one write; 40 flops on the common branch, 13 more in compressed cells.
- `calc_dt`: the first loop reads eight arrays and stores `dt_min`, the second loop reads `dt_min` back; 31 flops for the four limits.
- `pdv`: the predictor reads nine arrays, the corrector eleven (it adds the level-1 velocities), three writes each; 33 flops.
- `accelerate`: eight reads, two writes; 37 flops. `flux_calc`: two loops of three reads and one write; five flops each.
- `revert` and `reset_field`: one read, one write and one write-allocate per field, for two and four fields; no flops.
- `advec_cell` (two calls per step): the volume loop reads two or three arrays and writes `pre_vol` and `post_vol`; the flux loop reads four and writes two; the update loop reads six and writes six, four of which are temporaries nothing else reads. 45 flops.

- `advec_mom` (four calls per step): six loops, from the volume loop (two or three reads, two writes) to the velocity update (four reads, one write); the upwind loop alone has 18 of the 39 flops.
- `field_summary`: six reads into reductions, every tenth step; about 22 flops.

Multiplied by the calls per step these give the 451 flop and 2186 B per cell and step of Section 2.4, which the DDR counters confirmed to 1%.

3.3 Byte reductions

Each change is shown on the baseline code; the comments say what is inefficient and what the change does, the new code follows in Section 3.5. In the C snippets `i` abbreviates the node-sized index `FTNREF2D(j,k,x_max+5,...)` of cell (j, k), `iy` the cell-sized one, and `i+1` the neighbour $j + 1$; `[(j,k-1)]` writes a stencil neighbour out in full.

(1) `advec_mom`: direction-only helper loops once per sweep. The kernel is called four times per step (two sweeps × `xvel`, `yvel`). Its first four loops depend only on the sweep direction, not on the velocity component, and are recomputed for `yvel` with identical results:

```
! advec_mom_driver.f90 (baseline): one kernel call per velocity component
CALL advec_mom_kernel_c(..., xvel1, mass_flux_x, vol_flux_x, mass_flux_y, vol_flux_y, &
                        volume, density1, work_array1..6, celldx, celldy,             &
                        which_vel=1, sweep_number, direction)
CALL advec_mom_kernel_c(..., yvel1, ... same arguments ...,                           &
                        which_vel=2, sweep_number, direction)
! Inside the kernel, loops 1-4 (post_vol, node_flux, node_mass_post, node_mass_pre)
! use only direction and sweep_number: the yvel call rebuilds the four arrays over
! the haloed grid with exactly the values the xvel call produced, i.e. it streams
! mass_flux, vol_flux, volume, density1 and the four work arrays a second time.
! Only loop 5 (mom_flux from the limited upwind velocity) and loop 6 (the vel1
! update) read vel1. Change: split the kernel in a preparation part called once
! per sweep and a velocity part called once per component.
```

(2) `advec_cell`: temporaries in registers. The update loop of each sweep writes four work arrays and reads them back in the same iteration; nothing else reads them:

```
/* advec_cell_kernel_c.c (baseline), update loop of the x sweep */
pre_mass[i]  = density1[i]*pre_vol[i];                          /* stored here ...        */
post_mass[i] = pre_mass[i] + mass_flux_x[i] - mass_flux_x[i+1]; /* ... and reloaded here  */
post_ener[i] = (energy1[i]*pre_mass[i] + ener_flux[i] - ener_flux[i+1])
               / post_mass[i];
advec_vol[i] = pre_vol[i] + vol_flux_x[i] - vol_flux_x[i+1];
density1[i]  = post_mass[i] / advec_vol[i];
energy1[i]   = post_ener[i];
/* pre_mass, post_mass, post_ener and advec_vol are never read outside this
 * iteration: each costs a full-mesh store plus its read-for-ownership, four
 * arrays of DRAM traffic for values that fit in four registers. Change: make
 * them scalars; the arithmetic and its order stay the same. */
```

(3) Dead stores. `post_vol` in the first sweep of `advec_cell` and `pre_vol` in `advec_mom` are written over the whole haloed grid and never read:

```
/* advec_cell_kernel_c.c (baseline), sweep 1 (x direction) */
pre_vol[i]  = volume[i] + (vol_flux_x[i+1]-vol_flux_x[i])
                        + (vol_flux_y[iy+1]-vol_flux_y[iy]);
post_vol[i] = pre_vol[i] - (vol_flux_x[i+1]-vol_flux_x[i]);   /* no loop of sweep 1 reads
                                                                 post_vol: dead store */
/* advec_mom_kernel_c.c (baseline), every sweep */
post_vol[i] = volume[i] + vol_flux_y[iy+1] - vol_flux_y[iy];
pre_vol[i]  = post_vol[i] + vol_flux_x[i+1] - vol_flux_x[i];  /* no kernel reads pre_vol; the
                                                                 original source keeps it "for
                                                                 fidelity with the Fortran
                                                                 method" */
/* Each dead store is a full-mesh write plus a read-for-ownership over the haloed
 * grid, once per call. Change: delete both lines (and the pre_vol argument). */
```

(4) Reset and Revert without copies. `reset_field` copies the four end-of-step fields (`density1`, `energy1`, `xvel1`, `yvel1`) into their level-0 partners, 118 MB each; `revert` copies `density0`/`energy0` back into the level-1 arrays before the PdV corrector because the predictor wrote into them:

```
/* reset_field_kernel_c.c (baseline): one of four identical copy loops */
#pragma omp for private(j)
for (k=y_min;k<=y_max;k++)
#pragma omp simd
  for (j=x_min;j<=x_max;j++)
    density0[i] = density1[i];  /* 118 MB read, 118 MB written, 118 MB read-for-ownership;
                                   the same for energy, xvel and yvel. No kernel needs a
                                   field at a particular address: the fields are
                                   allocatable components of the tile. Change: exchange
                                   the allocations instead of the data. */
```

```
! PdV.f90 (baseline), end of the predictor pass
IF ( predict ) THEN
  CALL revert()   ! density1 = density0, energy1 = energy0 over the whole tile, because
                  ! the predictor wrote density1/energy1. The C corrector recomputes both
                  ! from the level-0 fields over the same range and nothing in between
                  ! reads the level-1 values. Change: skip the call on the C path.
ENDIF
```

(5) `calc_dt`: fused reduction. The kernel stores the local time step of every cell to `dt_min` and reduces the array in a second loop:

```
/* calc_dt_kernel_c.c (baseline) */
#pragma omp for private(...)
for (k...) for (j...) {
    ... the four limits dtct, dtut, dtvt, dtdivt of cell (j,k) ...
    dt_min[i] = MIN(dtct, MIN(dtut, MIN(dtvt, dtdivt)));  /* full-mesh store plus its
                                                             read-for-ownership */
}
#pragma omp for reduction(min:dt_min_val)
for (k...) for (j...)
    if (dt_min[i] < dt_min_val) dt_min_val = dt_min[i];    /* full-mesh reload of a value that
                                                             is only ever min-reduced */
/* The second pass exists only to separate the arithmetic from the reduction. The
 * cell and the criterion of the minimum, needed in the failure message, were not
 * tracked either: the C port returned cell (1,1) and "sound". Change: reduce inside
 * the first loop, drop dt_min, locate the cell afterwards and only when the step
 * is too small. */
```

(6) Streaming stores. Section 2.4 showed that a streaming store moves no read-for-ownership bytes on this CPU while the C kernels' stores do, and that the Fortran build already avoids most of them. Two routes are open: making the store streams of a loop recognisable to the core's write-

streaming detector, or explicit non-temporal stores. The pure-output loops are the candidates; the velocity update of `advec_mom` is not, because it loads `vel1[i]` in the same iteration and the line is in L1 when it is stored:

```
/* advec_mom_kernel_c.c (baseline), node_flux loop, direction 1 */
#pragma omp for private(j)
for (k=y_min;k<=y_max+1;k++)
#pragma omp simd
  for (j=x_min-2;j<=x_max+2;j++)
    node_flux[i] = 0.25*(mass_flux_x[(j,k-1)] + mass_flux_x[(j,k)]
                         +mass_flux_x[(j+1,k-1)] + mass_flux_x[(j+1,k)]);
/* node_flux is written once per iteration, contiguously in j, and never loaded in
 * this loop: the pattern the write-streaming detector needs to skip the
 * read-for-ownership. The same holds for post_vol, node_mass_post, node_mass_pre
 * and mom_flux, and for the flux arrays of advec_cell. GCC has no portable
 * non-temporal store builtin for AArch64 (stnp would be inline assembly) and it
 * cannot be validated on the development machine. Change: keep every output loop
 * in this shape after (1)-(3) and let the DDR-controller counters show whether
 * the hardware drops the read. */
```

The no-write-allocate model is only an upper bound for this item; the counters decide what the hardware grants.

3.4 Time-only changes

After the bytes shrink, four items move the kernels off the memory roof. **(T1)** `nowait` between independent worksharing loops: 12.5% of the cycles of a 128-thread run are barrier time, and many consecutive loops of a kernel touch disjoint arrays, or the same rows under the same static schedule. **(T2)** `-fno-math-errno`: `calc_dt`, `ideal_gas` and `viscosity` run entirely scalar because GCC keeps `sqrt` as a library call that may set `errno`. **(T3)** a vectorisable upwind loop: the advection kernels choose the upwind, donor and downwind indices by a branch on the sign of the flux and then load through those indices; loading all candidates and selecting the value removes the branch, and GCC additionally needs `-fno-trapping-math` to evaluate the limiter's divisions speculatively under the select. **(T4)** tile-level parallelism (one fork/join per tile instead of per loop) is the fallback if `nowait` is not enough. (T2) and (T3) change only the compile command of the C kernels:

```
# before: the project's GNU flags for the C kernels (Makefile, CFLAGS_GNU and OMP_GNU)
mpicc -O3 -march=native -funroll-loops -fopenmp -c kernels/*_kernel_c.c
# after: two flags added to CFLAGS_GNU (and CFLAGS_ARM); the Fortran flags are unchanged
#   -fno-math-errno     T2: sqrt() need not set errno, so GCC emits the NEON fsqrt
#                           instead of a scalar libm call (calc_dt, ideal_gas, viscosity)
#   -fno-trapping-math  T3: divisions may be evaluated speculatively, so the
#                           select-based upwind loops if-convert and vectorise
mpicc -O3 -march=native -funroll-loops -fno-math-errno -fno-trapping-math -fopenmp \
      -c kernels/*_kernel_c.c
# CloverLeaf enables no floating-point traps, so neither flag changes what the code asks
# for; the vector code GCC then generates does round differently somewhere, and the
# clover_bm.in gate detects it (Section 4). On the baseline the flags buy no time and no
# bytes; on the optimised kernels 3.6 % of time (ladder table). Withdrawn, see Section 4.
```

3.5 Implementation status

All byte changes are implemented; (T2) was implemented and withdrawn again (Section 4); (T1) and (T3) are implemented in `advec_mom` and still open in `advec_cell`; (6) is left to the hardware and (T4) is not started. `advec_mom`, `calc_dt` and the compiler flags are commits `59cdd3b` and

88fce34 of branch M3_dev; advec_cell, Reset and Revert are commit ff83119 of the same branch on the remote (the two lines are merged before the measurement). Every build of this ladder (base = the baseline source and flags, flags = 59cdd3b, cell = ff83119, mom = 88fce34, opt = the merge) was measured in one job on one node (job 34938, cn17, 2026-10-15) at 64×2, five repetitions under the DDR-controller counters (Table 5). The compiler flags alone change neither bytes nor time (+0.7%, −0.2%). The advec_cell line (temporaries, dead store, field swap) removes 12.1% of the traffic and 12.3% of the time, the advec_mom line (helper hoist, dead store, fused calc_dt reduction) 14.9% and 15.7%; merged they remove 27.8% of the bytes (2.828 to 2.043 TB) against the 29.3% the byte model predicted (Section 3.3), and the wall clock follows the bytes (−27.0%, 11.31 to 8.25 s) because every build sustains 248–252 GB/s. All builds print PASSED on bm3840; the clover_bm.in gate (jobs 34937, 34999 and 35002) is discussed in Section 4. A sixth build, optnf (commit 8d7d81f: opt with the baseline compiler flags, the kernel changes untouched), was measured in the same way in job 35003 (cn16, 2026-10-19) against a base rerun of its own (11.32 s): 8.56 s and 2.052 TB, −27.4% bytes and −24.3% time; it is the build Section 4 reports, for the reason given there. Below, the new code of every change; the comments say what was done.

(1) advec_mom split — implemented.

```
! advection.f90: the direction-only part runs once per sweep and tile, before the
! two per-component calls
DO tile=1,tiles_per_chunk
  CALL advec_mom_prep_driver(tile,direction,sweep_number) ! C only: advec_mom_prep_c builds
                                                          ! node_flux, node_mass_post,
                                                          ! node_mass_pre and post_vol
                                                          ! (work_array1/2/3/6)
  CALL advec_mom_driver(tile,xvel,direction,sweep_number) ! advec_mom_vel_c(xvel1, ...)
  CALL advec_mom_driver(tile,yvel,direction,sweep_number) ! advec_mom_vel_c(yvel1, ...)
ENDDO
! advec_mom_vel_c receives only vel1, the three node arrays, mom_flux (work_array4),
! celldx, celldy and direction; mass_flux, vol_flux, volume, density1 and pre_vol
! (work_array5) are no longer passed. Nothing between the three calls touches the
! work arrays. The Fortran kernel is untouched: the prep driver does nothing for
! use_fortran_kernels.
```

(2) advec_cell temporaries — implemented.

```
/* advec_cell_kernel_c.c, update loop of the x sweep (y sweep identical with the y fluxes) */
#pragma omp for private(j)
for (k=y_min;k<=y_max;k++) {
#pragma omp simd
  for (j=x_min;j<=x_max;j++) {
    /* the four temporaries live in registers: no store, no read-for-ownership */
    double pre_mass_s  = density1[i]*pre_vol[i];
    double post_mass_s = pre_mass_s + mass_flux_x[i] - mass_flux_x[i+1];
    double post_ener_s = (energy1[i]*pre_mass_s + ener_flux[i] - ener_flux[i+1]) / post_mass_s;
    double advec_vol_s = pre_vol[i] + vol_flux_x[i] - vol_flux_x[i+1];
    density1[i] = post_mass_s / advec_vol_s;   /* same operations in the same order as  */
    energy1[i]  = post_ener_s;                 /* before: the results are bit-identical */
  }
}
/* The pre_mass, post_mass, advec_vol and post_ener arguments stay in the Fortran
 * interface but are no longer touched. */
```

(3) Dead stores — implemented.

```
/* advec_cell_kernel_c.c, sweep 1: only pre_vol is computed, the post_vol store is gone */
pre_vol[i] = volume[i] + (vol_flux_x[i+1]-vol_flux_x[i]) + (vol_flux_y[iy+1]-vol_flux_y[iy]);

/* advec_mom_kernel_c.c, advec_mom_prep_c: post_vol only; pre_vol and its argument are
 * gone. IDXN/IDXC are the node- and cell-sized FTNREF2D index macros of the file. */
#pragma omp for private(j) nowait   /* T1: the node_flux loop that follows does not read
                                       post_vol; the barrier after node_flux protects
                                       node_mass_post, the first reader (k-1 stencil) */
for (k=y_min-2;k<=y_max+2;k++) {
#pragma omp simd
  for (j=x_min-2;j<=x_max+2;j++)
    post_vol[IDXN(j,k)] = volume[IDXC(j,k)] + vol_flux_y[IDXC(j,k+1)] - vol_flux_y[IDXC(j,k)];
}
```

(4) Reset and Revert — implemented.

```
! reset_field.f90, C path: the level-1 fields become the level-0 fields by exchanging
! the allocations; reset_field_kernel_c is no longer called
DO tile=1,tiles_per_chunk
  CALL swap_field(chunk%tiles(tile)%field%density0, chunk%tiles(tile)%field%density1)
  CALL swap_field(chunk%tiles(tile)%field%energy0,  chunk%tiles(tile)%field%energy1)
  CALL swap_field(chunk%tiles(tile)%field%xvel0,    chunk%tiles(tile)%field%xvel1)
  CALL swap_field(chunk%tiles(tile)%field%yvel0,    chunk%tiles(tile)%field%yvel1)
ENDDO

SUBROUTINE swap_field(a,b)              ! three MOVE_ALLOCs: O(1), no memory traffic
  REAL(KIND=8), ALLOCATABLE, INTENT(INOUT) :: a(:,:), b(:,:)
  REAL(KIND=8), ALLOCATABLE :: t(:,:)
  CALL MOVE_ALLOC(a,t); CALL MOVE_ALLOC(b,a); CALL MOVE_ALLOC(t,b)
END SUBROUTINE swap_field

! PdV.f90: the C corrector rewrites density1/energy1 over the whole range revert would
! copy, so the C path skips the copy; the Fortran kernels keep it
IF ( predict .AND. .NOT. use_C_kernels ) THEN
  CALL revert()
ENDIF
```

(5) `calc_dt` fused reduction — implemented.

```
/* calc_dt_kernel_c.c: one pass; dt_min is never written (the argument stays for the
 * Fortran interface). The four limits moved to an always_inline helper so that the hot
 * loop and the diagnostic scan share one definition. */
#pragma omp for private(j,dtct,dtut,dtvt,dtdivt) reduction(min:dt_min_val)
for (k=y_min;k<=y_max;k++) {
#pragma omp simd private(dtct,dtut,dtvt,dtdivt) reduction(min:dt_min_val)
  for (j=x_min;j<=x_max;j++) {
    cell_dt_limits(j,k, ..., &dtct,&dtut,&dtvt,&dtdivt);  /* sound, x-vel, y-vel, divergence */
    dt_min_val = MIN(dt_min_val, MIN(dtct,MIN(dtut,MIN(dtvt,dtdivt))));
  }                   /* every thread and vector lane keeps a private minimum; min is
                         exact, so the order in which OpenMP combines them is irrelevant */
}
if (dt_min_val < min_dt) {        /* failure path only: serial, runs once before the abort. */
  for (k...) for (j...) {         /* Re-evaluate the limits and report the first cell that  */
    cell_dt_limits(...);          /* reaches the minimum and which of the four it was,      */
    if (MIN(dtct,MIN(dtut,MIN(dtvt,dtdivt))) == dt_min_val) {  /* instead of the fixed      */
      j_ldt=j; k_ldt=k; dtl_control=...; goto found;           /* (1,1)/"sound" of the port */
    }
  }
}
```

(6) Streaming stores — not implemented explicitly. No non-temporal store was added. The restructured loops of (1), (3) and (T3) leave every output array of `advec_mom_prep_c` and

`mom_flux` written once per iteration, contiguously and without a load from the same lines, the shape asked for in (6); whether the write-streaming detector then drops the read-for-ownership is read from the DDR-controller counters of the optimised run.

(T1) `nowait` **— implemented in** `advec_mom`**, open in** `advec_cell`**.**

```
/* advec_mom_prep_c, direction 1: node_mass_post -> node_mass_pre without a barrier */
#pragma omp for private(j) schedule(static) nowait  /* same k range and schedule(static) as
                                                       the next loop: OpenMP guarantees the
                                                       same row -> thread mapping, and
                                                       node_mass_pre reads node_mass_post
                                                       only in its own row */
for (k=y_min;k<=y_max+1;k++)
#pragma omp simd
  for (j=x_min-1;j<=x_max+2;j++)
    node_mass_post[IDXN(j,k)] = 0.25*(density1[IDXC(j  ,k-1)]*post_vol[IDXN(j  ,k-1)]
                                     +density1[IDXC(j  ,k  )]*post_vol[IDXN(j  ,k  )]
                                     +density1[IDXC(j-1,k-1)]*post_vol[IDXN(j-1,k-1)]
                                     +density1[IDXC(j-1,k  )]*post_vol[IDXN(j-1,k  )]);
#pragma omp for private(j) schedule(static)         /* node_flux finished at its own barrier */
for (k=y_min;k<=y_max+1;k++)
#pragma omp simd
  for (j=x_min-1;j<=x_max+2;j++)
    node_mass_pre[IDXN(j,k)] = node_mass_post[IDXN(j,k)]
                             - node_flux[IDXN(j-1,k)] + node_flux[IDXN(j,k)];
/* Also nowait: the post_vol loop (see (3)) and, in direction 1 of advec_mom_vel_c, the
 * mom_flux loop before the velocity update (same rows, same schedule). Direction 2
 * keeps its barriers because the stencils cross rows. advec_cell is still all-barrier. */
```

(T2) Compiler flags — implemented in the Makefile for the GNU and ARM flag sets, as shown in Section 3.4.

(T3) Branch-free upwind loop — implemented in `advec_mom`**, open in** `advec_cell`**.**

```
/* advec_mom_vel_c, mom_flux loop, direction 1 (direction 2 is the same along k with celldy) */
#pragma omp for private(j) schedule(static) nowait
for (k=y_min;k<=y_max+1;k++) {
#pragma omp simd
  for (j=x_min-1;j<=x_max+1;j++) {
    const double nf  = node_flux[IDXN(j,k)];
    const int    neg = (nf < 0.0);   /* flow in -j: upwind j+2, donor j+1, downwind j */
    const double v_m1 = vel1[IDXN(j-1,k)], v_0  = vel1[IDXN(j  ,k)],  /* all four candidates: */
                 v_p1 = vel1[IDXN(j+1,k)], v_p2 = vel1[IDXN(j+2,k)];  /* contiguous loads     */
    const double v_up  = neg ? v_p2 : v_m1;      /* selects replace the branch and the      */
    const double v_don = neg ? v_p1 : v_0;       /* indirect loads through upwind/donor/    */
    const double v_dwn = neg ? v_0  : v_p1;      /* downwind                                */
    const double m_don = neg ? node_mass_pre[IDXN(j+1,k)] : node_mass_pre[IDXN(j,k)];
    const double width = celldx[FTNREF1D(j,x_min-2)];
    const double w_dif = neg ? celldx[FTNREF1D(j+1,x_min-2)] : celldx[FTNREF1D(j-1,x_min-2)];
    const double sigma   = fabs(nf)/m_don;
    const double vdiffuw = v_don - v_up,  vdiffdw = v_dwn - v_don;
    const double auw = fabs(vdiffuw), adw = fabs(vdiffdw);
    const double wind = (vdiffdw <= 0.0) ? -1.0 : 1.0;
    const double lim  = wind*MIN(width*((2.0-sigma)*adw/width+(1.0+sigma)*auw/w_dif)/6.0,
                                 MIN(auw,adw));  /* always evaluated (-fno-trapping-math) */
    const double limiter = (vdiffuw*vdiffdw > 0.0) ? lim : 0.0;  /* ... then selected */
    mom_flux[IDXN(j,k)] = (v_don + (1.0-sigma)*limiter)*nf;
  }
}
/* Same arithmetic in the same order as the branchy original: bit-identical results.
 * With -fno-trapping-math GCC vectorises all 13 loops of the file (vectoriser report,
 * GCC 13, x86); the NEON share on the cluster is measured with PAPI. */
```

(T4) Tile-level parallelism — not implemented.

build	changes	wall (s) median (min–max)	DRAM (TB)	bytes vs base (%)	time vs base (%)
base	M3 baseline source and CFLAGS (measured in job 34797)	11.31 (11.24–11.33)	2.828	+0.0	+0.0
flags	T2 only: -fno-math-errno -fno-trapping-math (no kernel change)	11.28 (11.27–11.34)	2.846	+0.7	−0.2
cell	flags + advec_cell register temporaries, dead post_vol store, reset/revert by field swap	9.92 (9.84–9.94)	2.486	−12.1	−12.3
mom	flags + advec_mom helper hoist, dead pre_vol store, calc_dt fused reduction (not cell)	9.54 (9.54–9.62)	2.405	−14.9	−15.7
opt	all changes (merge of cell and mom)	8.25 (8.23–8.33)	2.043	−27.8	−27.0
optnf	opt with the baseline CFLAGS (no -fno-math-errno -fno-trapping-math)	8.56 (8.53–8.63)	2.052	−27.4	−24.3

Table 5: Per-change attribution at 64×2 (bm3840, five repetitions under the DDR-controller counters, one node, one job): the commit ladder of `perf/variants.conf`, each build against the baseline build of the same job (job 34938; the `optnf` row from job 35003 against its own base rerun, 11.32 s). Filled from `data/ladder.csv` (`perf/jobs/m3_opt.sbatch`).

4 Results

Every optimised build is measured exactly like the baseline (Section 2.2): five repetitions under the DDR-controller counters, at the same five layouts, the correctness gate on `clover_bm.in` and `PASSED` on bm3840. Table 6 compares every layout with its own baseline; Figure 2 shows the wall clock against the memory floor.

Correctness. Job 34937 ran the `clover_bm.in` gate for every build of the ladder and job 34999 for the ladder's own baseline binary (Table 1). The baseline build reproduces the archived C value, 0.0057%, at every layout and with THP off, and the Fortran value: the reference stands, now from a build of known provenance. The four C builds with the compiler flags, the flags-only build included, report one and the same other deviation, 0.038%, at every layout, and the Fortran path of the optimised build reproduces its reference to all digits. The kernel changes are therefore neutral with respect to each other and to the flags-only build, and the change of the result is caused by `-fno-math-errno -fno-trapping-math` alone (the flags-only commit differs from the baseline in these two flags and in comments). The flags make GCC vectorise the `sqrt` kernels and if-convert the upwind loops, and the vector code rounds differently somewhere; the mechanism, e.g. a different multiply-add contraction under GCC's default `-ffp-contract=fast`, was not isolated. Both values are physically irrelevant and both fail CloverLeaf's own 0.001% threshold, but the gate of Section 2.2 is reproduction to 10^{-6}%, and the optimised build as measured does not pass it. The remedy is to drop them: `optnf`, the five byte changes with the baseline flags, reproduces 0.0057% at every layout and with THP off, and the Fortran value (job 35002, 7 of 7), and keeps the ladder's bytes (2.052 against 2.043 TB). It is the build reported below. What the flags were worth is now measured too: nothing on the baseline (+0.7% bytes, −0.2% time, Table 5) and 3.6% of the wall clock on the optimised kernels (8.25 against 8.56 s at the same bytes), almost all of it in the two `sqrt` kernels, whose vectorised square root only shows once the bytes around them shrink (`calc_dt` 0.45 against 0.62 s, `ideal_gas` 0.45 against 0.54 s; jobs 34939 and 35004). That is the price of reproducing the reference to 10^{-6}%; we pay

layout	baseline wall (s)	TB	GB/s	optimised wall (s)	TB	GB/s	speed-up
64x2	11.32 (11.24–11.33)	2.83	250	8.55 (8.53–8.63)	2.05	240	1.32
1x128	26.91 (17.44–29.20)	2.99	111	17.84 (12.87–23.16)	2.12	119	1.51
4x32	13.00 (12.76–13.21)	2.87	221	9.79 (9.63–9.94)	2.05	210	1.33
32x4	11.46 (11.41–11.47)	2.79	244	8.70 (8.69–8.77)	2.01	231	1.32
128x1	11.66 (11.63–11.77)	2.95	253	8.81 (8.79–8.82)	2.15	244	1.32

Table 6: C kernels, bm3840: baseline (job 34797) against the optimised build `optnf` (job 35003) at every layout, THP as installed. Speed-up = baseline median / optimised median.

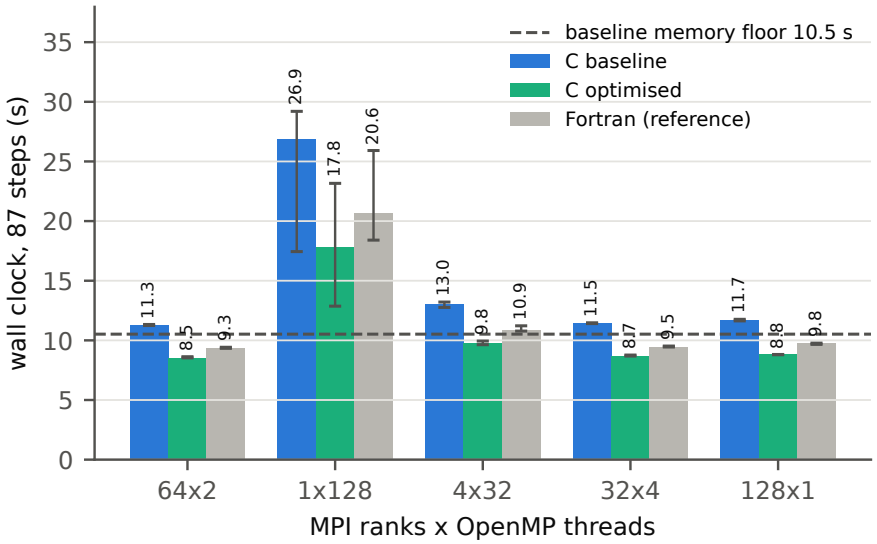


Figure 2: Wall clock per layout (median, bars min–max): C baseline, C optimised where measured, Fortran for reference. The dashed line is the memory floor of the baseline byte count at 266.5 GB/s.

it, because the gate is this project's definition of correct, and keep the flags as a separately measured option that a user with a looser criterion can switch on. All builds print `PASSED` on bm3840. (Builds on the x86 development machine gave 0.038% for every source, the baseline included; that cross-architecture comparison was invalid and is withdrawn.)

The gain is the byte saving and nothing else: every layout loses the same quarter of its traffic and speeds up by the same factor, 1.32–1.33× at the four balanced layouts and 1.51× at the NUMA-bound 1×128 (Table 6), the ladder adds up change by change (Table 5), and the counters confirm the byte model of Section 3.3 to within two points. The optimised C build is now faster than the Fortran reference at every layout (Figure 2): the C/Fortran byte gap of Section 2.5 is closed and reversed. What did not change is the character of the code. Figure 3 shows every kernel still on the bandwidth roof with IPC and back-end stall share as before (Table 7); the whole application moved up the roof, not off it, and sits at 90% of the hardware ceiling (240 GB/s) with 11% of the wall clock left above the memory floor of its own byte count (7.70 s). The two advection kernels took the whole gain (1.57× and 1.46×) and Reset and Revert vanished; `calc_dt` and `ideal_gas` gain 6–7% without their vectorised square root; PdV, `accelerate`, `flux_calc` and `viscosity` are untouched and are the next bytes to remove. The time-only items (T1 and T3 in `advec_cell`, T4, and the withdrawn flags) cannot buy more than that remaining 11% at this layout; the streaming-store lever of Section 3.3 can.

kernel	time (s)			IPC		back-end stall (%)	
	base	optnf	ratio	base	optnf	base	optnf
advec_mom	4.19	2.67	1.57	0.27	0.28	91	89
advec_cell	2.42	1.66	1.46	0.28	0.37	91	88
pdv_correct	0.71	0.72	0.99	0.20	0.20	92	93
calc_dt	0.66	0.62	1.06	0.50	0.53	81	80
pdv_predict	0.61	0.66	0.92	0.22	0.21	92	92
ideal_gas	0.58	0.54	1.07	0.31	0.32	87	87
accelerate	0.56	0.61	0.93	0.27	0.26	91	91
flux_calc	0.39	0.39	1.00	0.16	0.16	95	95
reset_field	0.38	–	–	0.10	–	96	–
viscosity	0.32	0.33	0.96	0.99	0.94	62	64
revert	0.19	–	–	0.10	–	96	–
update_halo	0.04	0.04	1.08	0.57	0.61	42	41
field_summary	0.03	0.04	0.86	0.42	0.38	85	87

Table 7: Per-kernel PAPI counters at 64×2, THP on: baseline (job 34798) against the optimised build `optnf` (job 35004). Time is the region time of the ipc event set (max over threads); Reset and Revert have no C region any more (field swap). The two halves of the split `advec_mom` count under one region.

5 Conclusions (draft)

- The C kernels are memory-bound (0.2 flop/B against a 5 flop/B ridge) and the 64×2 baseline already sustains 94% of the 266.5 GB/s hardware ceiling over the whole run: only fewer bytes can make it faster.
- The analytic byte model matches the counters to 1% and predicts the five planned changes at −29% of the traffic (floor 10.5 to 7.4 s); the read-for-ownership that the C stores pay, and that streaming stores and the Fortran build avoid, is a sixth lever worth up to another −23%.
- Measured: the five byte changes remove 27.4% of the DRAM traffic (model: 29.3%) and the wall clock follows, 1.32× at 64×2 and 1.32–1.51× at every other layout; the optimised C build is faster than the Fortran reference everywhere and still sits on the bandwidth roof, 11% above the memory floor of its own bytes.
- The correctness gate: the rebuilt baseline binary reproduces the archived `clover_bm.in` reference at every layout (job 34999), and so does the optimised build with the baseline flags (job 35002). The two compiler flags `-fno-math-errno -fno-trapping-math` alone move the result; they are worth nothing on the baseline and 3.6% on the optimised kernels, and were dropped: a measured trade-off between time and reproducibility, decided for the gate.

References

[1] Jan Laukemann, Georg Hager, Gerhard Wellein, et al. CloverLeaf on Intel multi-core CPUs: A case study in write-allocate evasion. arXiv:2311.04797, 2023.

[2] A. C. Mallinson, D. A. Beckingsale, W. P. Gaudin, J. A. Herdman, J. M. Levesque, and S. A. Jarvis. CloverLeaf: Preparing hydrodynamics codes for exascale. In *Proceedings of the Cray User Group (CUG)*, 2013.

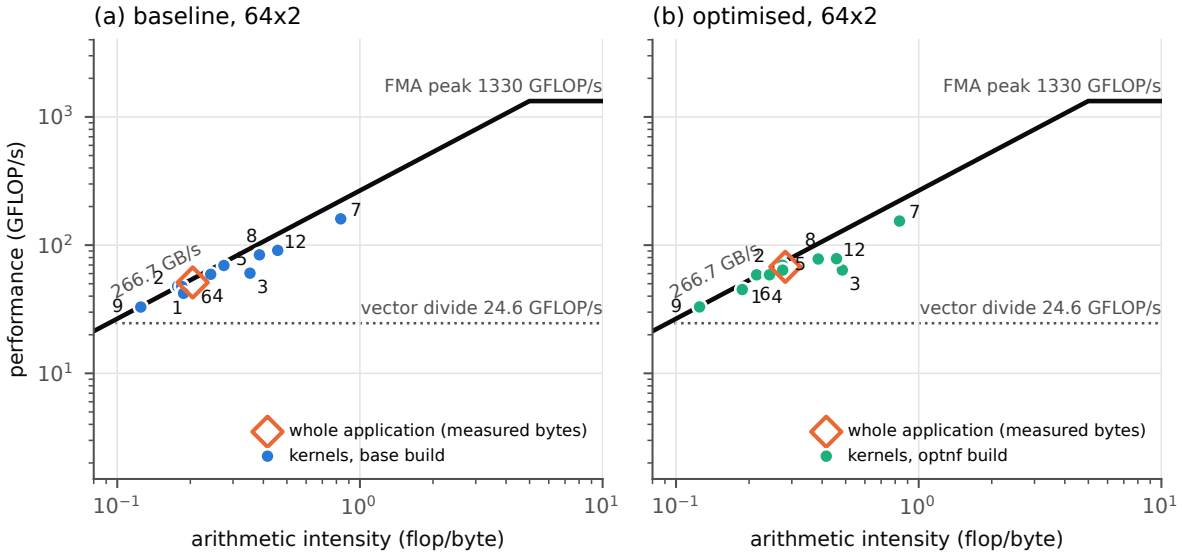


Figure 3: Node roofline at 64×2, baseline (a) against the optimised build (b): kernels from their PAPI region times (jobs 34798 and 35004) and the byte model of each build, numbered as in Table 2; the hollow diamond is the whole application from the measured DRAM bytes and wall clock.